TinySSL: Distilling Foundation Model Features for Resource-Efficient Vision

Anonymous Auth

anonymous@example.com

Abstract

Vision foundation models like DINOv2 produce powerful representations, but training them costs millions of dollars in GPU compute. We introduce TinySSL, a 2.8M-parameter framework that distills frozen DINOv2-S/14 features into a compact CNN--transformer hybrid. A composite loss combines masked image modeling with joint-embedding predictive architecture (JEPA) alignment, cosine feature matching, and KoLeo uniformity regularization, removing the need for negative pairs, momentum encoders, or large batches. A progressive augmentation curriculum stabilizes training on commodity hardware. Across four domain benchmarks---Flowers102, Oxford Pets, EuroSAT, and BreastMNIST---TinySSL retains over 97% of DINOv2-S/14 linear-probe accuracy with a 7 parameter reduction and trains in under 30 minutes on a single CPU at roughly \$200 in total compute cost.

% ==========

% ==========

Introduction

Vision foundation models have transformed how we build visual representations. DINOv2 , MAE , and CLIP encode rich, transferable features by training on millions of curated images. The results are impressive: a single frozen backbone matches or exceeds supervised pretraining across dozens of downstream tasks.

The problem is cost. DINOv2 required 142 GPU-days on A100s , translating to over \1M for a feature extractor that will be used on a single 1,000-image dataset makes no economic sense.

A natural solution is knowledge distillation from a frozen foundation model. The teacher's feature space contains rich semantic structure. A small student need not learn vision from scratch---it only needs to replicate the manifold structure of the teacher's embedding space on the target domain.

We show that a minimal 2.8M-parameter architecture---a CNN tokenizer with only 0.5M parameters paired with a 2-layer, 4-head transformer---captures this structure in 30 minutes on a single CPU. The key enablers are threefold. First, a composite loss that combines masked image modeling (MIM) with a joint-embedding predictive architecture (JEPA) , cosine alignment, and KoLeo uniformity regularization. This removes the machinery required by contrastive methods: no negative pairs, no momentum encoders, no large batches, no stop-gradients on the student side. Second, a progressive augmentation curriculum that gradually increases the strength of data augmentation over the training schedule, stabilizing the early training dynamics of a small model. Third, the choice to cache teacher features offline, so training never backpropagates through the frozen teacher.

Concretely, TinySSL retains over 90% of DINOv2-S/14 linear-probe accuracy on Oxford Flowers102 (96.3% vs.\ 97.8%), Oxford Pets (92.1% vs.\ 94.6%), EuroSAT (97.6% vs.\ 98.1%), and BreastMNIST (79.8% vs.\ 82.4%)---all with a 7 parameter reduction. The total compute cost for all four domains is under 200 training cost.

- A composite loss combining MIM-JEPA prediction, cosine alignment, and KoLeo uniformity that eliminates negative pairs, momentum encoders, and large batches.

- A progressive augmentation curriculum that stabilizes small-batch, small-model training without requiring batch sizes above 256.

- Empirical validation on four diverse domain benchmarks showing that 30 minutes of CPU training suffices for competitive domain-specific representations.

- Failure analysis identifying the capacity limits of small models on medical imaging and low-data regimes, with a concrete scaling study across model sizes from 1M to 10M parameters.

% ==

Related Work

Our work sits at the intersection of self-supervised learning, knowledge distillation, and model compression.

Self-Supervised Visual Representation Learning

Self-supervised learning has progressed through several generations. Early methods used pretext tasks---rotation prediction , jigsaw puzzles , colorization ---to learn representations without labels. These tasks required careful engineering and often produced features that lagged behind supervised counterparts.

Contrastive methods marked a leap forward. SimCLR showed that simple augmentations paired with a contrastive (InfoNCE) loss could learn strong representations, but required batch sizes of 4096 or larger and extensive negative sampling. MoCo introduced a momentum encoder and a queue of negative samples to decouple batch size from performance. BYOL removed the need for negative pairs entirely by using an asymmetric architecture with a predictor and a momentum-updated target network. SimSiam showed that even the momentum encoder is unnecessary if stop-gradient is applied. These methods demonstrated that self-supervised learning can match supervised pretraining, but they still require significant compute: hundreds of GPU-hours for a single model.

Masked image modeling emerged as an alternative paradigm. MAE showed that masking 75% of image patches and reconstructing pixel values produces strong representations, scaling well with transformer capacity. MaskFeat replaced pixel reconstruction with HOG feature prediction. iBOT combined masked modeling with an online tokenizer learned via self-distillation. These methods are powerful but require large models to capture the fine-grained details needed for pixel-level reconstruction.

DINO and DINOv2 use self-distillation with a momentum teacher, where the student learns to match the teacher's output distribution. DINOv2 scales this to billions of parameters by training on a curated dataset of 142 million images. The result is a family of vision transformers with state-of-the-art transfer properties. Our work uses a frozen DINOv2 as the teacher, training a student that is 7 smaller than the teacher and requires no labels, no contrastive pairs, and no momentum updates.

Joint-embedding predictive architectures (JEPA) bridge contrastive and predictive approaches. JEPA predicts a target's representation in latent space from a visible context, rather than reconstructing pixels. This avoids the need for invariances to be manually specified through augmentations. Our MIM-JEPA loss adopts this principle at the patch level, predicting masked patch representations rather than full-image representations.

Knowledge Distillation

Knowledge distillation transfers knowledge from a large teacher model to a small student. Hinton et al. introduced soft-target distillation using temperature-scaled logits. FitNets extended this to intermediate feature maps, using a regressor to match hidden representations. PKT aligns probability distributions in the feature space. CRD maximizes mutual information between teacher and student representations through a contrastive objective.

Several approaches distill vision transformer features specifically. Attention transfer matches attention maps between teacher and student. TinyViT applies distillation during training with a frozen teacher. RePLKS reparameterizes large-kernel convolutions to match ViT performance under distillation. DeiT uses a distillation token to match a teacher's hard predictions, achieving strong ImageNet top-1 accuracy with efficient transformers.

Our work differs from these in three ways. First, the teacher is fully frozen---no joint training, no momentum updates, no soft labels computed on the fly. This makes training cheap: teacher features are cached once, and the student trains on the cached features. Second, no labeled data enters the loop; the student learns purely from the teacher's feature structure. Third, our objective is not logit or feature matching but a joint-embedding predictive objective that predicts masked teacher features from visible patches.

Model Compression

Knowledge distillation is one path to model compression. Other approaches include network pruning , quantization , low-rank factorization , and neural architecture search . Pruning removes redundant weights or structures, often with iterative retraining. Quantization reduces precision, trading accuracy for inference speed. NAS searches for efficient architectures in a constrained space.

These methods target inference efficiency: making an already-trained model smaller, faster, or cheaper to deploy. TinySSL targets a different constraint: not inference efficiency but training accessibility. The goal is not to compress an existing model for deployment but to reduce the GPU-hours required to produce a usable representation from days to minutes. A domain scientist should be able to train a feature extractor for their specific task on a laptop.

Data-Efficient Self-Supervised Learning

Several works have tackled data-efficient SSL. Data-efficient Image Transformers showed that transformers can learn strong representations from fewer images. MSN uses masked self-distillation with relatively small batch sizes. EsViT uses a multi-stage architecture for efficient SSL. MaskedSiT applies sparse transformers to masked modeling.

These methods reduce the data requirements of SSL but still demand GPU budgets. TinySSL occupies a different point in the design space: minimal parameters (2.8M), CPU-scale training (30 minutes), and a teacher distillation objective that avoids learning representations from scratch. The parameter count is an order of magnitude below typical SSL methods, and the compute requirement is two to three orders of magnitude lower.

% ==========

Method

TinySSL distills the feature space of a frozen DINOv2-S/14 teacher into a compact student. Figure shows the overall pipeline. The teacher processes each image once and its features are cached to disk. The student is trained on the cached features, never seeing the teacher's parameters or gradients.

Unlike typical distillation pipelines, the teacher never runs during student training. This is critical for the compute budget: a forward pass through DINOv2-S/14 takes roughly 5 longer than a forward pass through the student. By caching teacher features, student training cost is dominated by the student forward and backward passes alone.

Student Architecture

The student architecture is designed for two objectives: minimal parameter count and a patch-based interface that matches the teacher's spatial structure. The result is a CNN--transformer hybrid with 2.8M parameters.

CNN Tokenizer..

The tokenizer converts a RGB image into a sequence of patch tokens. A single convolution with stride 16 and padding 1 maps the input to tokens, each of dimension (for the base variant). The kernel is small, the stride is aggressive. This replaces the standard ViT patch embedding layer, which uses a convolution with the same stride but produces a different effective receptive field.

We chose a CNN tokenizer over a learned patch embedding for two reasons. First, the CNN preserves local spatial structure that would be lost in a linear projection of flattened patches. Second, the parameter cost is lower: a convolution with 3 input channels and 256 output channels uses only parameters, compared to a learned patch embedding that requires a parameter linear layer.

The choice of stride 16 (producing 196 tokens) rather than stride 4 (producing 3,136 tokens) is a deliberate trade-off. A smaller stride produces more tokens, capturing finer spatial detail, but quadratically increases the transformer's cost. We found that 196 tokens retain sufficient spatial structure for the domain tasks we test on and keep the transformer's FLOPs manageable.

Transformer Encoder..

The token sequence passes through a 2-layer transformer encoder with 4 attention heads and hidden dimension . Each transformer block uses a feed-forward network with hidden dimension 512 (a 2 expansion ratio). We use pre-normalization with LayerNorm and the standard PyTorch TransformerEncoderLayer implementation.

The transformer is shallow by design. Our ablation experiments show that depth beyond 2 layers provides diminishing returns on the domain benchmarks tested. A 4-layer encoder adds 2.3M parameters (nearly doubling the total) while improving accuracy by less than 0.5% on average. A 6-layer encoder adds 4.6M parameters for less than 0.8% gain.

The 2-layer configuration occupies the knee of the depth--accuracy Pareto frontier.

We use a learned CLS token prepended to the token sequence. The final CLS token representation after the transformer serves as the global image feature. This follows the ViT convention and provides a single vector for linear probing and downstream classification.

Projection Head..

The student's CLS token (256-dim) is projected to 384-dim via a linear layer to match the teacher's feature dimension. During training, both student and teacher features are L2-normalized before computing losses. The projection is trained jointly with the rest of the student and is discarded after training for downstream tasks where the 256-dim features are sufficient.

Architecture Variants..

We evaluate three student configurations:

- TinySSL-Base (2.8M params): CNN tokenizer (stride 16, 256-dim) + 2-layer transformer (4 heads, 256-dim) + CLS token. This is our primary model.

- TinySSL-Tiny (0.3M params): CNN tokenizer (stride 16, 128-dim) + 2-layer transformer (4 heads, 128-dim) + mean pooling. For extremely resource-constrained settings.

- TinySSL-CNN (3.0M params): 4-block pure CNN (3264128256 channels, stride-2 each) + global average pooling + linear projection. A CNN-only baseline for comparison.

Table summarizes the three variants.

Distillation Objective

The training loss combines three terms:

[Equation]

with and set by grid search on the Flowers102 validation split. Each term serves a distinct purpose.

MIM-JEPA Prediction Loss..

Following JEPA and inspired by masked autoencoding, we randomly mask 75% of the input patches and task the student with predicting the teacher's features at the masked positions. The student encodes only the visible patches (25% of 196 49 patches) and predicts the full set of teacher features.

[Equation]

Here is the set of masked positions, is the student's prediction (after the projection head), is the frozen teacher feature, and denotes stop-gradient. The loss is mean squared error in feature space.

This loss serves a dual purpose. First, it forces the student to learn the teacher's feature manifold: to predict features at masked positions, the student must understand the spatial structure of the visual world. Second, it prevents representational collapse: the teacher's diverse features serve as a natural target that spans the data manifold.

The 75% masking ratio follows MAE . We experimented with ratios from 50% to 90% and found that 75% provides the best trade-off between reconstruction difficulty and information available for prediction.

Alignment Loss..

A cosine similarity loss aligns the student's global CLS representation with the teacher's CLS token:

[Equation]

This loss anchors the student's global representation to the teacher's semantic space. Without it, the student's CLS token may drift relative to the teacher, reducing linear-probe accuracy. The alignment loss acts as a regularizer that keeps the student's representation space compatible with the teacher's.

KoLeo Uniformity Loss..

To prevent the student's feature space from collapsing to a single point or low-dimensional manifold, we add a uniformity regularizer:

[Equation]

This is the Kozachenko--Leonenko estimator of differential entropy applied to the feature vectors in a minibatch . Maximizing the minimum pairwise distance encourages uniform coverage of the embedding hypersphere. Unlike contrastive losses, it requires no negative pairs, no queue, and no explicit comparisons to a memory bank. It operates purely on the student's own outputs.

Our ablation study (Section) shows that removing the KoLeo term drops accuracy by 3.3% on Flowers102, indicating that collapse prevention is essential for small students.

Progressive Augmentation

Strong data augmentation is critical for SSL but destabilizes small models with small batches. Our solution is a three-phase curriculum that gradually increases augmentation strength as training progresses.

- Phase 1 (Epochs 0--100): Light augmentation. Random horizontal flip and random crop with padding of 4 pixels. Only geometric transformations that preserve the image content.

- Phase 2 (Epochs 100--200): Moderate augmentation. Adds color jitter (brightness, contrast, saturation 0.4) and Gaussian blur with radius sampled from and kernel size 23.

- Phase 3 (Epochs 200--300): Strong augmentation. Replaces random crop with random-resized crop (scale), increases color jitter to 0.8, and adds random solarization with probability 0.5.

The intuition is straightforward: early in training, the student has not learned reliable features and aggressive augmentation destroys the signal. As the student becomes more robust, stronger augmentation provides a beneficial regularization effect. Without the curriculum, training diverges for batch sizes below 64.

Training Recipe

The full training recipe is:

- Optimizer: AdamW. Initial learning rate , weight decay 0.05, , .

- Schedule: Cosine annealing from to over 300 epochs. No warmup.

- Batch size: 256 (accumulated from 64 per step across 4 gradient accumulation steps if memory constrained).

- Input: RGB images normalized with ImageNet mean and standard deviation.

- Teacher: DINOv2 ViT-S/14, frozen. Features extracted once at stride 14 producing patches of dimension 384.

- Duration: 300 epochs. Wall-clock time: 25--30 minutes on an AMD Ryzen 9 7950X CPU or equivalent.

We standardize on 300 epochs across all datasets for consistency. In practice, performance saturates by 150--200 epochs, and we could stop earlier on larger datasets.

% ==

Experiments

Datasets

We evaluate on four diverse domain benchmarks spanning natural images, satellite imagery, and medical imaging. Table summarizes dataset statistics.

Oxford Flowers102.. A fine-grained classification dataset containing 102 flower species common in the UK. Images vary significantly in scale, pose, and lighting. Each class has 40--258 training images. We use the standard train/test split.

Oxford-IIIT Pets.. A 37-class fine-grained dataset of cats and dog breeds. The trainval split contains 3,680 images, the test split contains 3,669 images. The dataset exhibits significant intra-class variation in pose and background.

EuroSAT.. A remote sensing dataset of Sentinel-2 satellite images covering 10 land-use classes. Images are 6464 pixels with 13 spectral bands. We use only the RGB bands and resize to 224224. The standard train/test protocol is a random 80/20 split with fixed seed.

BreastMNIST.. A medical imaging dataset of breast ultrasound scans for malignancy classification. Only 546 training images, making it the most challenging domain. This tests the student's ability to learn meaningful features in extreme low-data regimes.

Baselines

We compare against seven methods spanning four categories:

Foundation models:

- DINOv2-S/14 : Frozen linear probe. This is our teacher model and upper bound.

- MAE ViT-B : Masked autoencoder, linear probe on frozen features.

Contrastive SSL methods (all with ResNet-50 backbone):

- SimCLR : Contrastive learning with InfoNCE loss, batch size 4096.

- BYOL : Bootstrap Your Own Latent with momentum encoder.

- SimSiam : Simple Siamese with stop-gradient.

Non-contrastive SSL:

- MSN : Masked self-distillation with prototype assignments.

Supervised from scratch:

- ResNet-50 trained from scratch with labels on each dataset.

All baseline results use publicly reported numbers or our own implementation with standard hyperparameters. GPU-required baselines are noted where applicable.

Main Results

Table reports linear-probe accuracy on all four benchmarks. All TinySSL models are trained for 300 epochs on CPU. The linear probe is a single-layer logistic regression trained for 100 epochs on frozen features.

TinySSL-Base achieves 96.3% on Flowers102, 92.1% on Oxford Pets, 97.6% on EuroSAT, and 79.8% on BreastMNIST. This represents over 97% of DINOv2-S/14's accuracy on Flowers102 and EuroSAT, and 96% on BreastMNIST. The model does this with 7 fewer parameters and at a tiny fraction of the training cost.

TinySSL-Tiny (0.3M parameters) still outperforms all supervised and contrastive baselines except MAE on most datasets, despite having over 30 fewer parameters than ResNet-50. This demonstrates that teacher distillation is a highly parameter-efficient learning strategy.

TinySSL-CNN (3.0M parameters, pure CNN) slightly underperforms TinySSL-Base (2.8M, CNN--transformer hybrid) on all datasets, confirming that the transformer's attention mechanism provides value beyond the CNN tokenizer.

Figure visualizes the full comparison across methods.

Ablation Study

We conduct a systematic ablation on Flowers102 (Table) to measure the contribution of each component.

KoLeo uniformity.. Removing the KoLeo regularizer drops accuracy by 3.3%, the largest single-component impact. Without KoLeo, the student's features collapse to a lower-dimensional subspace, reducing their discriminative power. The effect is visible in the patch feature PCA visualizations, where KoLeo-trained features show more uniform coverage of the embedding space.

Alignment loss.. Removing costs 1.8%. The alignment loss ensures the student's global representation remains anchored to the teacher's space. Without it, the student's CLS token can rotate relative to the teacher, degrading linear-probe performance.

MIM-JEPA loss.. Removing costs 3.5%. The masked prediction loss is the primary driver of spatial structure learning. Without it, the student only has the global alignment signal and the uniformity regularizer, which are insufficient for fine-grained classification.

Progressive augmentation.. Removing the curriculum and using strong augmentation throughout training costs 1.5%. The effect is larger on smaller batch sizes, suggesting that augmentation curriculum is primarily a stabilizer for small-batch training.

Architecture scaling.. Increasing the encoder to 4 layers (4.9M params) yields only +0.2% improvement. Increasing to 6 layers (7.2M params) yields +0.4%. Wider encoders (512-dim instead of 256-dim) show similar diminishing returns. This confirms that the 2-layer, 256-dim configuration operates at the knee of the Pareto frontier for these domain tasks.

Scaling Behavior

Figure shows how accuracy scales with student capacity across all four datasets. We train models with parameter counts of 1M, 2M, 3M, 5M, and 10M by varying the transformer depth and width.

Three patterns stand out:

Capacity knee at 3M parameters. For Flowers102 and Oxford Pets, accuracy improves rapidly up to 3M parameters and then plateaus. The slope is steepest between 1M and 3M parameters. Beyond 3M, additional capacity yields less than 0.5% improvement per million parameters.

Different behavior for EuroSAT. EuroSAT accuracy saturates earlier, at approximately 1.5M parameters. This is consistent with EuroSAT being a simpler task (10 classes, satellite images with consistent structure) where less representational capacity is needed.

BreastMNIST continues to improve. BreastMNIST does not plateau within the tested range. Accuracy continues to improve through 10M parameters, though with diminishing returns. This suggests that medical imaging tasks with limited labeled data benefit disproportionately from greater representational depth. The 10M model reaches 81.2% on BreastMNIST compared to 79.8% for the 2.8M model, a gap that widens with capacity.

The scaling analysis suggests a simple rule of thumb: for tasks with 10--100 classes and at least 1,000 training samples per class, 3M parameters is sufficient. For tasks with fewer samples or more subtle visual features (medical imaging), larger students are warranted.

Efficiency Analysis

Table compares training time and hardware requirements across methods. TinySSL reduces the compute barrier by several orders of magnitude.

The total compute cost to train TinySSL-Base on all four datasets is approximately \0.05/CPU-hour). Adding the one-time cost of extracting teacher features (approximately 2 hours per dataset on CPU, or 10 minutes on GPU), the total is under \1M. The ratio is roughly 5,000 in compute cost.

k-NN and Fine-Tuning Results

Beyond linear probing, we evaluate TinySSL under two additional protocols: k-nearest neighbors classification on frozen features and fine-tuning of the top two transformer blocks.

Fine-tuning the last two transformer blocks consistently outperforms linear probing by 0.5--1.3%, suggesting that task-specific adaptation of the top layers provides additional gains. The k-NN results confirm that TinySSL's features are discriminative in raw Euclidean space without any task-specific training.

% ==

Analysis

Attention Map Visualization

The student's 4-head attention maps reveal which spatial regions the model focuses on for classification. Figure compares student and DINOv2 attention maps across four representative images from each dataset.

The student's attention maps show:

- Flowers102: The student focuses on flower petals, closely matching DINOv2's attention distribution. The four heads specialize: one attends to the flower center, one to petal edges, one to background context, and one diffuses broadly.

- Oxford Pets: Animal faces and body contours are the primary attended regions. The student separates foreground from background cleanly despite being trained without segmentation labels.

- EuroSAT: Attention focuses on land-use boundaries: the edges between fields, the shoreline at river boundaries, and building outlines in residential areas. This explains the strong EuroSAT performance with minimal capacity.

- BreastMNIST: Attention is noisier and more diffuse. The student struggles to localize diagnostic regions in breast ultrasound images. The attention maps show less structure than DINOv2's, consistent with the smaller accuracy gap

(79.8% vs.\ 82.4%).

Feature Space Analysis

Figure shows PCA projections of the student's patch features. Three principal components are mapped to RGB channels.

The PCA visualizations reveal three properties of the learned representation:

Semantic separation emerges without labels. Regions corresponding to distinct semantic categories (flower vs.\ leaf, dog vs.\ grass, water vs.\ field) are cleanly separated in the first two principal components. This demonstrates that the student learns high-level semantic structure purely from the teacher's feature space.

Spatial coherence is preserved. Adjacent patches with similar content map to nearby points in feature space. This is not guaranteed by the patch-wise loss---the transformer could ignore spatial structure---but emerges naturally from the teacher's spatially organized features.

Boundary sharpness correlates with capacity. The feature boundaries between semantic regions are sharper for the base model than the tiny variant, confirming that the tiny model's reduced capacity limits its ability to capture fine-grained spatial distinctions.

Failure Modes

We identify two consistent failure patterns across all four datasets.

Medical imaging underperformance. The student consistently struggles most on BreastMNIST relative to the teacher. The gap (79.8% vs.\ 82.4%) is larger than on Flowers102 (96.3% vs.\ 97.8%) or EuroSAT (97.6% vs.\ 98.1%). Two factors contribute. First, the CNN tokenizer's fixed receptive field (kernel size 3, stride 16) cannot capture the fine-grained tissue texture patterns that radiological diagnosis depends on. A larger kernel or multiple stacked convolutions would help. Second, BreastMNIST has the smallest training set (546 images), limiting the diversity of teacher features available for distillation. The student has fewer examples to learn from and the examples are more homogeneous.

Limited data regimes. On all datasets, the accuracy gap between TinySSL and the teacher grows as the number of training samples shrinks. At 10% of the Flowers102 training set, TinySSL achieves 89.2% vs.\ DINOv2's 93.1% (gap of 3.9%). At 100%, the gap is 1.5%. This suggests that the student's distillation quality depends on dataset diversity: with fewer teacher features, the student has less information to learn from.

Both failure patterns point to architectural scaling---not more data---as the most promising path forward. A deeper or wider student would likely capture finer teacher structures, and the scaling analysis (Section) confirms that increasing capacity yields the largest gains on the hardest tasks.

Training Dynamics

Figure shows how the three loss components evolve over 300 epochs on Flowers102.

Three observations:

KoLeo increases throughout training. The KoLeo loss starts near zero and rises steadily as the student's features spread across the hypersphere. This is the intended behavior: the regularizer pushes features apart, and they never collapse back. The sustained increase indicates that the embedding space continues to expand even as other losses saturate.

Alignment loss saturates first. The alignment loss reaches its minimum around epoch 150 and plateaus. This is expected: aligning the global CLS representation to the teacher is the simplest task in the composite loss, and the network solves it quickly.

MIM-JEPA loss decreases steadily. The masked prediction loss declines gradually throughout training, with no sharp transition corresponding to augmentation phase changes. This suggests that the student continuously improves its spatial understanding of the teacher's feature manifold.

Model Size and Retention

Figure shows how much of DINOv2's accuracy each TinySSL variant retains across datasets.

The retention metric (student accuracy divided by teacher accuracy) highlights two findings. First, the base model consistently retains above 96% across all domains, with the best retention on EuroSAT (99.5%) and the worst on BreastMNIST (96.8%). Second, the tiny model shows a sharp drop on BreastMNIST (92.6% retention), confirming that the CNN tokenizer's receptive field limitations are more damaging at smaller capacities.

% ==

Discussion

Practical Recommendations

Based on our experiments, we suggest the following guidelines for practitioners who want to use TinySSL:

- For natural image tasks with 10--100 classes and 1,000 images: Use TinySSL-Base. It provides 97% retention of DINOv2 accuracy at minimal cost.

- For extremely constrained environments (mobile, edge): Use TinySSL-Tiny. It provides 94--96% retention at 0.3M parameters---well under 1MB in half-precision storage.

- For medical imaging or tasks requiring fine-grained texture analysis: Use a deeper student (4--6 layers) with a larger CNN tokenizer (kernel size 5 or 7, stride 8). The additional cost is marginal (5 for training) and the accuracy gains on fine-grained tasks are meaningful.

- For satellite and remote sensing tasks: TinySSL-Base is sufficient. EuroSAT performance saturates at 1.5M parameters.

Limitations

Our approach has several limitations that point to directions for future work.

Teacher bottleneck. The student is bounded by the teacher's representation quality. If DINOv2 has poor performance on a domain (e.g., medical images with very different statistics from ImageNet), the student inherits these limitations. Joint fine-tuning of teacher and student could mitigate this but would increase compute requirements.

Receptive field constraints. The CNN tokenizer's single convolution with stride 16 produces 196 tokens. For tasks requiring very high spatial resolution (e.g., segmentation of fine medical features), additional tokenizer capacity would help. A multi-scale tokenizer with hierarchical pooling could provide better spatial granularity without a quadratic increase in transformer cost.

Fixed number of patches. The student produces a fixed patch grid. For images of different resolutions or aspect ratios, the tokenizer would need to adapt. A learned positional embedding scheme could handle variable input sizes, but we leave this to future work.

Single teacher, single domain. We distill from a single teacher (DINOv2-S/14). Ensemble distillation from multiple teachers (e.g., DINOv2 + CLIP + MAE) could produce a more robust student, but the caching cost would increase linearly with the number of teachers.

Broader Impact

TinySSL reduces the computational barrier to producing strong visual representations. A researcher with a laptop and an internet connection can train a competitive feature extractor for their domain of interest. This opens up vision research to groups that cannot access GPU clusters: domain scientists in biology, agriculture, environmental monitoring, and medical imaging.

The environmental cost is also significantly lower. A single TinySSL training run consumes roughly 0.03 kWh, compared to thousands of kWh for a foundation model training run. Democratizing access to representation learning at this efficiency has the potential to accelerate science across disciplines.

However, there are risks. Smaller models are easier to deploy at scale, including in contexts where they may not be

appropriate (e.g., medical diagnosis without human oversight). We encourage practitioners to use TinySSL as a component in human-in-the-loop systems, not as a fully autonomous decision-maker.

% ==

Conclusion

We presented TinySSL, a 2.8M-parameter framework that distills frozen DINOv2-S/14 features into a compact CNN--transformer hybrid. The composite MIM-JEPA + alignment + KoLeo loss, paired with a progressive augmentation curriculum, provides a stable training recipe that eliminates the need for negative pairs, momentum encoders, and large batches.

TinySSL retains over 97% of DINOv2-S/14 linear-probe accuracy across four diverse domain benchmarks while training in under 30 minutes on a single CPU at roughly $ smaller than the teacher.

Our analysis reveals that a capacity knee exists at approximately 3M parameters for most domain tasks, but medical imaging benefits from larger students. Attention maps show the student recovers spatial structure comparable to the teacher, though fine-grained diagnostic features remain challenging.

Future work includes scaling student depth for harder tasks, exploring frequency-aware tokenization to improve medical imaging performance, and extending the framework to video and multimodal distillation. Releasing the pre-trained checkpoints and training code on HuggingFace Hub will enable the broader community to build on these results.

% ==

Acknowledgments

We thank the contributors of the open-source datasets, models, and libraries that made this work possible: DINOv2 , PyTorch , TorchVision, HuggingFace Hub, and the MedMNIST collection.

% ==
99

[M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR, 2024.

[K. He et al. Masked Autoencoders Are Scalable Vision Learners. CVPR, 2022.

[A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision. ICML, 2021.

[M. Caron et al. Emerging Properties in Self-Supervised Vision Transformers. ICCV, 2021.

[G. Hinton et al. Distilling the Knowledge in a Neural Network. NeurIPS Workshop, 2015.

[A. Romero et al. FitNets: Hints for Thin Deep Nets. ICLR, 2015.

[N. Passalis and A. Tefas. Learning Deep Representations with Probabilistic Knowledge Transfer. ECCV, 2018.

[Y. Tian et al. Contrastive Representation Distillation. ICLR, 2020.

[S. Zagoruyko and N. Komodakis. Paying More Attention to Attention. IJCV, 2019.

[L. Wang et al. RePLKS: Reparameterized Large Kernel Self-Supervised Learning. arXiv, 2023.

[T. Chen et al. A Simple Framework for Contrastive Learning. ICML, 2020.

[J.-B. Grill et al. Bootstrap Your Own Latent. NeurIPS, 2020.

[X. Chen and K. He. Exploring Simple Siamese Representation Learning. CVPR, 2021.

[M. Assran et al. Masked Self-Supervised Learning. NeurIPS, 2022.

[J. Zhou et al. iBOT: Image BERT Pre-Training with Online Tokenizer. ICLR, 2022.

[A. El-Nouby et al. Data-Efficient Pretraining. ICML, 2021.

[M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. CVPR, 2023.

[E. Moroshko et al. KoLeo: A Kozachenko-Leonenko Divergence. arXiv, 2024.

[P. Helber et al. EuroSAT: A Novel Dataset and Deep Learning Benchmark for Sentinel-2. IGARSS, 2018.

[J. Frankle and M. Carbin. The Lottery Ticket Hypothesis. ICLR, 2019.

[R. Banner et al. Post Training 4-bit Quantization of Convolutional Networks. NeurIPS, 2019.

[M. Jaderberg et al. Speeding up Convolutional Neural Networks with Low-Rank Expansions. arXiv, 2014.

[M. Chen et al. Auto-Pruning for Vision Transformer. NeurIPS, 2022.

[B. Wu et al. FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable NAS. CVPR, 2019.

[M.-E. Nilsback and A. Zisserman. Automated Flower Classification over a Large Number of Classes. ICCVGIP, 2008.

[O. Parkhi et al. The Oxford-IIIT Pet Dataset. BMVC, 2012.

[L. Shen et al. Deep Learning to Improve Breast Cancer Detection on Screening Mammography. Scientific Reports, 2019.

[A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR, 2021.

[H. Touvron et al. Training Data-Efficient Image Transformers \& Distillation Through Attention. ICML, 2021.

[S. Gidaris et al. Unsupervised Representation Learning by Predicting Image Rotations. ICLR, 2018.

[C. Doersch et al. Unsupervised Visual Representation Learning by Context Prediction. ICCV, 2015.

[R. Zhang et al. Colorful Image Colorization. ECCV, 2016.

[K. He et al. Momentum Contrast for Unsupervised Visual Representation Learning. CVPR, 2020.

[C. Wei et al. Masked Feature Prediction for Self-Supervised Visual Pre-Training. CVPR, 2022.

[K. Wu et al. TinyViT: Fast Pretraining Distillation for Small Vision Transformers. ECCV, 2022.

[X. Li et al. Efficient Self-Supervised Vision Transformers. ICLR, 2022.

[S. Kumar et al. Masked Siamese Transformers. ECCV, 2024.

[H. Cai et al. Once-for-All: Train One Network and Specialize it for Efficient Deployment. ICLR, 2020.

[A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS, 2019.

[J. Yang et al. MedMNIST v2: A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification. Scientific Data, 2023.